Capstone 3 — Domain C: Entity Resolution Agent

Build a ReAct agent that reasons through multi-step entity resolution — searching filings, computing fuzzy matches, cross-referencing registries, and merging entity profiles.

PREREQUISITES

Complete **M05 (Function Calling)**, **M06 (Multi-Tool Orchestration)**, **M12 (ReAct Agent Loop)**, and **M13 (Planning & Task Decomposition)** before starting this capstone. You should be comfortable defining tool schemas, handling `tool_use` / `tool_result` message flows, and building agentic loops that check `stop_reason`.

Project Brief

BUSINESS CONTEXT

Business names in UCC filings are notoriously inconsistent. The same company appears as "Acme Logistics LLC", "ACME LOGISTICS, L.L.C.", "Acme Logistics Company", and "Acme Logistics Inc." across different states. A human analyst looking at these four names must decide: are these all the same company, or four different companies?

The pain is scale and subtlety. A commercial data provider processes millions of filings. Manual entity resolution costs $2–5 per entity. At 100,000 entities per month, that is $200K–$500K annually in analyst time. Worse, humans are inconsistent — one analyst merges two entities, another keeps them separate, and the database has conflicting records.

Your agent automates this: given a business name, it searches filings across states, computes fuzzy match scores between candidates, cross-references official business registries, and produces a merged entity profile with a confidence score. The agent must REASON about each match — "ACME LOGISTICS, L.L.C." vs "Acme Logistics LLC" is clearly the same entity, but "Acme Logistics Inc." in a different state needs more investigation.

WHAT YOU WILL BUILD

A ReAct agent with 5 tools that implements the Thought-Action-Observation cycle for entity resolution:

- `search_filings_by_name` — find filing candidates across states
- `fuzzy_match_score` — compute similarity between name pairs
- `get_filing_details` — retrieve full filing data for comparison
- `get_business_registry_data` — cross-reference official SOS registrations
- `merge_entity_profile` — create a unified profile from matched entities

The key challenge: the agent must decide at each step what to do next based on what it just learned. If fuzzy match scores are high, proceed to merge. If they are ambiguous, gather more evidence from the registry. If they are low, mark as distinct. This branching logic is what makes it a reasoning agent, not a script.

Environment Setup

You need **Python 3.10+** or **Node.js 18+** and an Anthropic API key. Run these commands to create your project:

SETUP
SETUP

```
mkdir capstone-3-entity-resolution && cd capstone-3-entity-resolution
python -m venv venv && source venv/bin/activate   # Windows: venv\Scripts\Activate.ps1

# Pin dependencies for reproducibility
echo "anthropic>=0.40.0" > requirements.txt
pip install -r requirements.txt
export ANTHROPIC_API_KEY=your-key-here            # Windows: set ANTHROPIC_API_KEY=your-key-here
```

File Structure

PROJECT TREE

PROJECT TREE

```
capstone-3-entity-resolution/
├── entity_tools.py      # Fuzzy matching + mock data tools
├── entity_agent.py      # Entity resolution agent with ReAct loop
├── entity_tools.ts      # TypeScript tools
├── entity_agent.ts      # TypeScript agent
└── requirements.txt     # Dependencies
```

Domain Glossary

Entity Resolution

Determining whether two or more records refer to the same real-world entity. Also called record linkage, deduplication, or entity matching.

Fuzzy Matching

Comparing strings using approximate algorithms (Levenshtein, Jaro-Winkler, token sort) that return a similarity score rather than requiring exact equality.

Canonical Name

The "official" version of an entity's name chosen as the standard reference. Usually matches the SOS business registration.

Business Registry

The official state database of registered business entities (LLCs, corporations). Contains the legal entity name, formation date, status, and registered agent.

Confidence Score

A 0.0-1.0 number indicating how certain the agent is that two entities are the same. Above 0.9 = definite match. 0.7-0.9 = likely match. Below 0.7 = needs more evidence.

Token Sort Ratio

A fuzzy matching algorithm that sorts the words in both strings alphabetically before comparing. "Acme Logistics LLC" and "LLC Logistics Acme" would score 100% — order does not matter.

ReAct Architecture

The agent follows a Thought → Action → Observation loop. Each cycle, it reasons about what it knows, what it needs, and which tool to call next. Here is a typical resolution trace:

REACT REASONING TRACE — ENTITY RESOLUTION

ENTITY MERGE DECISION FLOW

Mock Data Specification

RESOLUTION REQUEST · FUZZY MATCH RESPONSE

RESOLUTION REQUEST

```
{
  "request_id": "ER-2024-0078",
  "input_name": "Acme Logistics LLC",
  "input_state": "DE",
  "candidates": [
    {"name": "ACME LOGISTICS, L.L.C.", "state": "DE", "filing_count": 3},
    {"name": "Acme Logistics Company", "state": "DE", "filing_count": 1},
    {"name": "Acme Logistics Inc.", "state": "NY", "filing_count": 2}
  ]
}
```

FUZZY MATCH RESPONSE

```
{
  "entity_a": "Acme Logistics LLC",
  "entity_b": "ACME LOGISTICS, L.L.C.",
  "scores": {
    "exact": 0.45,
    "normalized": 0.95,
    "token_sort_ratio": 0.98
  },
  "recommendation": "likely_match"
}
// Note: levenshtein and jaro_winkler are stretch metrics — install
// `rapidfuzz` (`pip install rapidfuzz`) and add them to the scores
// dict to enable. The base capstone uses three deterministic scores.
```

Implementation Phases

1. **Phase 1 — Mock Tools (60 min):** Implement all 5 tools with mock data covering 3+ entities, name variations, and edge cases (registry not found, conflicting data).
2. **Phase 2 — System Prompt (30 min):** Write a ReAct-style system prompt that instructs Claude to externalize reasoning, plan before acting, and adapt strategy based on intermediate results.
3. **Phase 3 — ReAct Loop (45 min):** Implement the agentic loop with `stop_reason` checking. The loop must handle 5+ tool calls in a single resolution and branch based on fuzzy match results.
4. **Phase 4 — Branching Logic (30 min):** Ensure the agent adapts: high confidence → merge directly. Ambiguous → gather more evidence from registry. Low confidence → mark as distinct.
5. **Phase 5 — Error Recovery (30 min):** Handle mid-chain failures: registry unavailable, timeout on filing search. Agent should proceed with partial data and lower confidence.
6. **Phase 6 — Testing (45 min):** Run 10 test cases verifying correct merges, correct separations, and proper handling of ambiguous/adversarial inputs.

Step 1: Create entity_tools.py

WHAT & WHY

What: You will create a file containing 5 mock tool functions that simulate searching UCC filings, computing fuzzy match scores, retrieving filing details, querying business registries, and merging entity profiles.

Why: Mock tools let you develop and test the agent's reasoning logic without calling real APIs. Every tool returns structured data with explicit error handling, so the agent always knows whether a call succeeded or failed and can adapt its strategy accordingly.

Create the file `entity_tools.py` and paste the following code:

PYTHON
PYTHON

```
# entity_tools.py — Mock tools for entity resolution
IMPORT re

# --- Tool 1: Search filings by name ---
MOCK_CANDIDATES = {
    "acme logistics llc": [
        {"name": "ACME LOGISTICS, L.L.C.", "state": "DE", "filing_count": 3, "most_recent": "2024-01-15"},
        {"name": "Acme Logistics Company", "state": "DE", "filing_count": 1, "most_recent": "2023-08-20"},
        {"name": "Acme Logistics Inc.", "state": "NY", "filing_count": 2, "most_recent": "2023-12-01"},
    ],
    "buildright construction": [
        {"name": "BuildRight Construction LLC", "state": "NY", "filing_count": 1, "most_recent": "2024-01-10"},
        {"name": "Build Right Construction", "state": "NY", "filing_count": 1, "most_recent": "2022-03-15"},
    ],

    # … (full source in the desktop HTML) …

                "total_lien_exposure": total_lien_exposure,
                "states": list(set([primary_entity.get("state", "")] + [c.get("state", "") for c in merge_candidates])),
                "confidence": confidence,
                "merge_log": [f"Merged '{c.get('name', '')}' (score: {c.get('match_score', 'N/A')})" for c in
merge_candidates]}
    CATCH:
        RETURN {"is_error": True, "error_category": "INTERNAL_ERROR", "is_retryable": True, "context": str(e)}
```

Quick verify — run this in your terminal:

VERIFY
VERIFY

```
python -c "from entity_tools import search_filings_by_name; print(search_filings_by_name('Acme Logistics LLC'))"
```

Expected output:

```
{'is_error': False, 'candidates': [{'name': 'ACME LOGISTICS, L.L.C.', 'state': 'DE', 'filing_count':
3, 'most_recent': '2024-01-15'}, {'name': 'Acme Logistics Company', 'state': 'DE', 'filing_count': 1,
'most_recent': '2023-08-20'}, {'name': 'Acme Logistics Inc.', 'state': 'NY', 'filing_count': 2,
'most_recent': '2023-12-01'}], 'total': 3}
```

Checkpoint: Step 1 Complete

You built 5 interconnected tools. The agent will call them in sequence based on its reasoning: first `search_filings_by_name` to find candidates, then `fuzzy_match_score` to compare each pair, then `get_business_registry_data` to verify matches with official records, and finally `merge_entity_profile` to create the unified profile. The fuzzy matching uses a simplified name normalization that strips legal suffixes (LLC, Inc, Corp) before comparing — because "Acme Logistics LLC" and "Acme Logistics Inc." are the same business name with different entity type designations.

TROUBLESHOOTING

If you see `ModuleNotFoundError: No module named 'entity_tools'` when testing the import, make sure you saved the file as `entity_tools.py` (not `entity-tools.py`) and your terminal is in the same directory as the file.

Step 2: Create entity_agent.py

WHAT & WHY

What: You will create the ReAct agent that defines tool schemas for Claude, writes a system prompt encoding entity resolution decision criteria, and implements the agentic loop that routes tool calls to your mock functions.

Why: This is the core of the capstone — the agent must reason about which tool to call next based on intermediate results. High fuzzy scores lead directly to merge; ambiguous scores trigger registry lookups for more evidence; low scores mark entities as distinct. The loop caps at 15 iterations to prevent runaway resolution chains.

Create the file `entity_agent.py` and paste the following code:

PYTHON

PYTHON

```
# entity_agent.py — ReAct Entity Resolution Agent
IMPORT anthropic, json
FROM entity_tools import (search_filings_by_name, fuzzy_match_score,
    get_filing_details, get_business_registry_data, merge_entity_profile)

SYSTEM_PROMPT = """You are an Entity Resolution Agent for a commercial data provider.
Given a business entity name, you determine whether UCC filing records
under different name variations refer to the same real-world company.

## Your Reasoning Process (ReAct)
For each resolution request, think step by step:
1. THINK: What do I know? What do I need to find out?
2. ACT: Call the appropriate tool to gather evidence.
3. OBSERVE: What did the tool return? What does it tell me?

    # … (full source in the desktop HTML) …


    RETURN "Resolution exceeded maximum iterations."

IF __name__ == "__main__":
    result = run_entity_agent("Resolve entity)
    print(result)
```

Checkpoint: Step 2 Complete

You built a ReAct entity resolution agent that: (1) searches for name variations using fuzzy matching, (2) computes similarity scores between each candidate pair, (3) cross-references official business registries for verification, and (4) merges confirmed matches into a unified profile with a confidence score. The system prompt encodes the decision criteria (what threshold = merge vs. investigate vs. distinct), and the agent adapts its strategy based on intermediate results. The safety cap is 15 iterations because entity resolution can require 8-12 tool calls for complex cases.

Step 3: Test the Entity Resolution Agent

WHAT & WHY

What: Run the agent end-to-end to resolve "Acme Logistics LLC" across states and observe the full ReAct reasoning trace.

Why: Testing confirms that the agentic loop correctly routes tool calls, that the agent reasons about fuzzy match scores before deciding to merge or separate, and that the final merged profile includes all expected data.

Run the agent from your terminal:

PYTHON

PYTHON

```
python entity_agent.py
```

Expected output (your exact wording will vary — the agent reasons in natural language):

```
I'll resolve the entity "Acme Logistics LLC" by searching for name variations across states.

[Agent calls search_filings_by_name, finds 3 candidates]
[Agent calls fuzzy_match_score for each candidate pair]
[Agent calls get_business_registry_data to verify DE and NY registrations]
[Agent calls merge_entity_profile for the confirmed DE matches]

Resolution complete:
- Canonical name: Acme Logistics LLC
- 2 DE entities merged (ACME LOGISTICS, L.L.C. + Acme Logistics Company) — confidence: 0.94
- 1 NY entity (Acme Logistics Inc.) kept separate — different registry, different address
- Total filings consolidated: 4 across Delaware
- Total lien exposure: $2,300,000
```

Checkpoint: Step 3 Complete

Your agent successfully resolved entity variations by reasoning through multiple tool calls. It merged the two Delaware entities (high fuzzy match + same registry) and correctly kept the New York entity separate (different address, different registry entry). The confidence score reflects the strength of the evidence gathered.

Verify Everything Works

Run this end-to-end verification to confirm all components work together:

END-TO-END TEST

END-TO-END TEST

```python
# e2e_test.py — Quick verification script
FROM entity_tools import (search_filings_by_name, fuzzy_match_score,
    get_filing_details, get_business_registry_data, merge_entity_profile)

# 1. Search works
result = search_filings_by_name("Acme Logistics LLC")
assert not result["is_error"], "Search failed"
assert result["total"] == 3, f"Expected 3 candidates, got {result['total']}"
print("Search:   OK — 3 candidates found")

# 2. Fuzzy matching works
score = fuzzy_match_score("Acme Logistics LLC", "ACME LOGISTICS, L.L.C.")
assert not score["is_error"], "Fuzzy match failed"
assert score["recommendation"] == "likely_match", f"Expected likely_match, got {score['recommendation']}"

    # … (full source in the desktop HTML) …

# 5. Error handling works
empty = search_filings_by_name("Nonexistent Corp XYZ")
assert empty["is_error"], "Expected error for unknown entity"
print(f"Errors:   OK — graceful handling for unknown entities")

print("\nAll checks passed. Run 'python entity_agent.py' for the full agent test.")
```

Expected output:

```
Search:   OK — 3 candidates found
Fuzzy:    OK — likely_match (token_sort: 1.0)
Registry: OK — Acme Logistics LLC in DE
Merge:    OK — Acme Logistics LLC, confidence: 0.94
Errors:   OK — graceful handling for unknown entities

All checks passed. Run 'python entity_agent.py' for the full agent test.
```

Testing Guide

TYPE	SCENARIO	EXPECTED BEHAVIOR
`Happy`	Two DE name variants (LLC vs L.L.C.)	Merges with high confidence (>0.9), cites same address
`Happy`	Three candidates: 2 match, 1 distinct	Merges 2 DE entities, keeps NY entity separate
`Happy`	Legal suffix difference only	High match score, quick merge
`Happy`	Entity with filings in multiple states	Cross-references registry to confirm single entity
`Happy`	Clean resolution, no conflicts	Merged profile with all filings consolidated
`Edge`	Similar names, different companies in same state	Distinguishes using address and registry data
`Edge`	Registry returns NOT_FOUND for one candidate	Proceeds with available data, lowers confidence
`Edge`	"possible_match" fuzzy score	Gathers additional evidence before deciding
`Adversarial`	Common name with 50+ candidates	Filters by state, limits scope, does not attempt all
`Adversarial`	Same name, same state, different file numbers	Flags conflict for human review, does not force merge

Troubleshooting

Common issues and how to fix them:

`ModuleNotFoundError: No module named 'anthropic'`

You have not installed the Anthropic SDK. Run `pip install anthropic` in your activated virtual environment. If you are using Node.js, run `npm install @anthropic-ai/sdk`.

`AuthenticationError: Invalid API key`

Your `ANTHROPIC_API_KEY` environment variable is not set or contains an invalid key. Verify with `echo $ANTHROPIC_API_KEY` (Linux/Mac) or `echo %ANTHROPIC_API_KEY%` (Windows). The key should start with `sk-ant-`. Re-export it if needed: `export ANTHROPIC_API_KEY=sk-ant-...`

`ImportError: cannot import name 'search_filings_by_name' from 'entity_tools'`

Both `entity_tools.py` and `entity_agent.py` must be in the same directory. Also check that you saved the tools file as `entity_tools.py` (underscores, not hyphens) and that it contains all 5 function definitions.

Agent loops without resolving (hits 15 iterations)

If the agent keeps calling tools without reaching a conclusion, check: (1) your `MOCK_CANDIDATES` data contains entries for the query you are testing, (2) the system prompt decision criteria thresholds are present, and (3) the `merge_entity_profile` tool is included in the `TOOLS` list so the agent can finalize its resolution. You can lower the iteration cap from 15 to 10 during debugging to fail faster.

Compliance & Regulatory Notes

ENTITY RESOLUTION STANDARDS

False positive risk: Merging two distinct entities into one creates incorrect risk profiles. A lender relying on a false merge might deny a loan to a clean company because it was conflated with a heavily-liened entity. Always prefer conservative merges with clear evidence.

FCRA implications: If merged entity profiles are used for credit decisions, accuracy requirements under the Fair Credit Reporting Act apply. Incorrect merges could trigger disputes and regulatory action.

Audit trail: Every merge decision must be logged with the evidence that supported it (match scores, registry data, address comparisons). The `merge_log` in the output serves this purpose.